

Python - Update Tuples

Updating Tuples in Python

In Python, **tuple** is an immutable sequence, meaning once a tuple is created, its elements cannot be changed, added, or removed.

To update a tuple in Python, you can combine various operations to create a new tuple. For instance, you can concatenate tuples, slice them, or use tuple unpacking to achieve the desired result. This often involves converting the tuple to a list, making the necessary modifications, and then converting it back to a tuple.

Updating Tuples Using Concatenation Operator

The concatenation operator in Python, denoted by `+`, is used to join two sequences, such as strings, lists, or tuples, into a single sequence. When applied to tuples, the concatenation operator joins the elements of the two (or more) tuples to create a new tuple containing all the elements from both tuples.

We can update a tuple using the concatenation operator by creating a new tuple that combines the original tuple with additional elements.

Since tuples are immutable, updating tuples using concatenation operator does not modify the original tuple but instead creates a new one with the desired elements.

Example

In the following example, we create a new tuple by concatenating "T1" with "T2" using the `+` operator –

```
# Original tuple
T1 = (10, 20, 30, 40)
# Tuple to be concatenated
T2 = ('one', 'two', 'three', 'four')
# Updating the tuple using the concatenation operator
T1 = T1 + T2
print(T1)
```

It will produce the following output –

```
(10, 20, 30, 40, 'one', 'two', 'three', 'four')
```

Updating Tuples Using Slicing

Slicing in Python is used to extract a portion of a sequence (such as a list, tuple, or string) by specifying a range of indices. The syntax for slicing is as follows –

```
sequence[start:stop:step]
```

Where,

- **start** is the index at which the slice begins (inclusive).
- **stop** is the index at which the slice ends (exclusive).
- **step** is the interval between elements in the slice (optional).

We can update a tuple using slicing by creating a new tuple that includes slices of the original tuple combined with new elements.

Example

In this example, we are updating a tuple by slicing it into two parts and inserting new elements between the slices –

```
# Original tuple
T1 = (37, 14, 95, 40)
# Elements to be added
```

```

new_elements = ('green', 'blue', 'red', 'pink')
# Extracting slices of the original tuple
# Elements before index 2
part1 = T1[:2]
# Elements from index 2 onward
part2 = T1[2:]
# Create a new tuple
updated_tuple = part1 + new_elements + part2
# Printing the updated tuple
print("Original Tuple:", T1)
print("Updated Tuple:", updated_tuple)

```

Following is the output of the above code –

```

Original Tuple: (37, 14, 95, 40)
Updated Tuple: (37, 14, 'green', 'blue', 'red', 'pink', 95, 40)

```

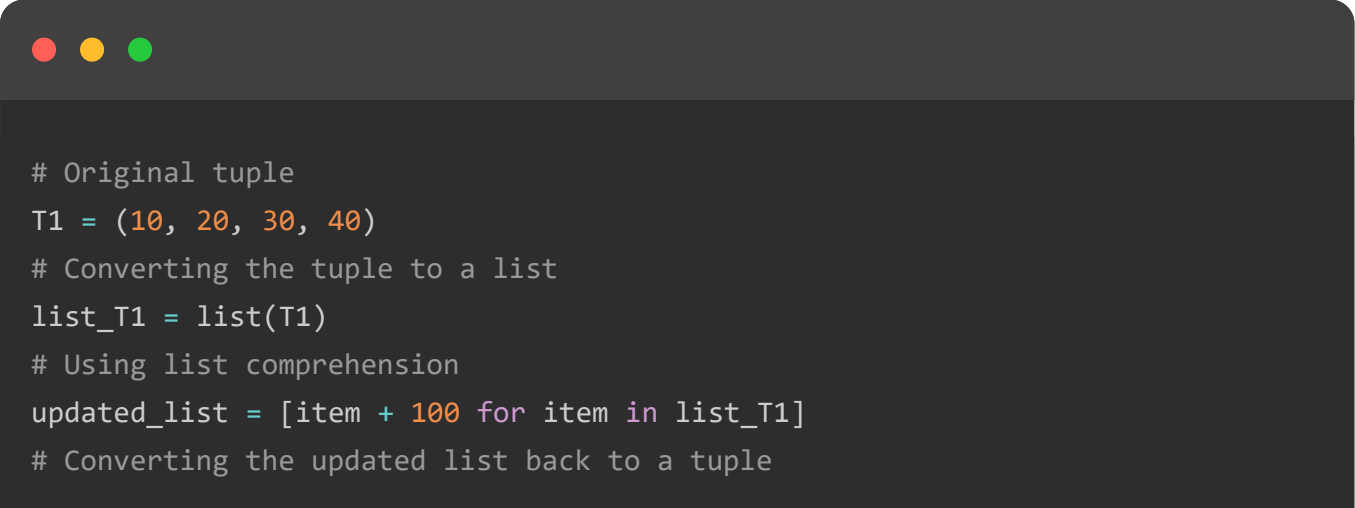
Updating Tuples Using List Comprehension

List comprehension in Python is a concise way to create lists. It allows you to generate new lists by applying an expression to each item in an existing iterable, such as a list, tuple, or string, optionally including a condition to filter elements.

Since tuples are immutable, updating a tuple involves converting it to a list, making the desired changes using list comprehension, and then converting it back to a tuple.

Example

In the example below, we are updating a tuple by first converting it to a list and using list comprehension to add 100 to each element. We then convert the list back to a tuple to get the updated tuple –



```

# Original tuple
T1 = (10, 20, 30, 40)
# Converting the tuple to a list
list_T1 = list(T1)
# Using list comprehension
updated_list = [item + 100 for item in list_T1]
# Converting the updated list back to a tuple

```

```
updated_tuple = tuple(updated_list)
# Printing the updated tuple
print("Original Tuple:", T1)
print("Updated Tuple:", updated_tuple)
```

Output of the above code is as follows –

```
Original Tuple: (10, 20, 30, 40)
Updated Tuple: (110, 120, 130, 140)
```

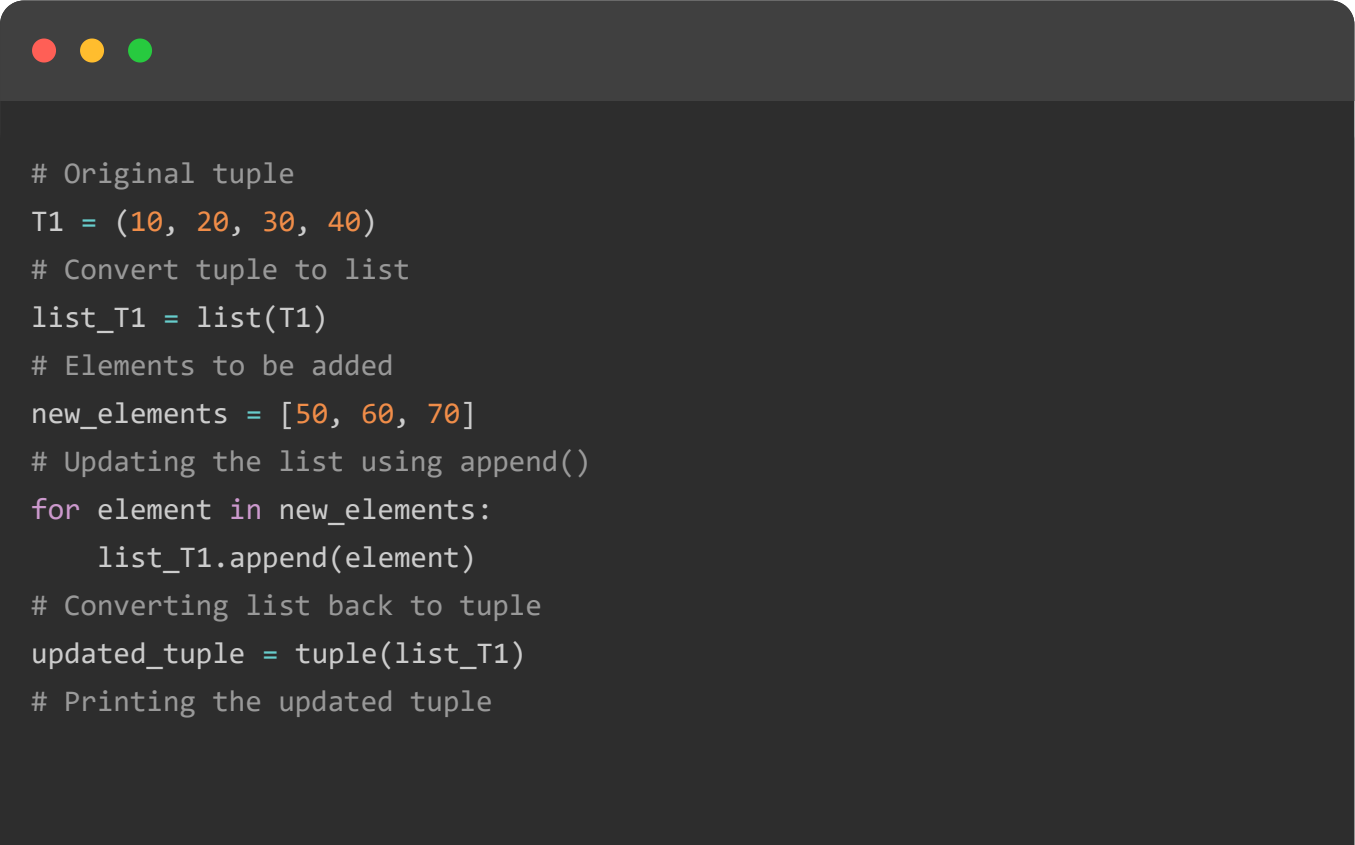
Updating Tuples Using append() function

The append() function is used to add a single element to the end of a list. However, since tuples are immutable, the append() function cannot be directly used to update a tuple.

To update a tuple using the append() function, we need to first convert the tuple to a list, then use append() to add elements, and finally convert the list back to a tuple.

Example

In the following example, we first convert the original tuple "T1" to a list "list_T1". We then use a loop to iterate over the new elements and append each one to the list using the append() function. Finally, we convert the updated list back to a tuple to get the updated tuple –



```
# Original tuple
T1 = (10, 20, 30, 40)
# Convert tuple to list
list_T1 = list(T1)
# Elements to be added
new_elements = [50, 60, 70]
# Updating the list using append()
for element in new_elements:
    list_T1.append(element)
# Converting list back to tuple
updated_tuple = tuple(list_T1)
# Printing the updated tuple
```

```
print("Original Tuple:", T1)
print("Updated Tuple:", updated_tuple)
```

We get the output as shown below –

```
Original Tuple: (10, 20, 30, 40)
Updated Tuple: (10, 20, 30, 40, 50, 60, 70)
```

TOP TUTORIALS

- Python Tutorial
- Java Tutorial
- C++ Tutorial
- C Programming Tutorial
- C# Tutorial
- PHP Tutorial
- R Tutorial
- HTML Tutorial
- CSS Tutorial
- JavaScript Tutorial
- SQL Tutorial

TRENDING TECHNOLOGIES

- Cloud Computing Tutorial
- Amazon Web Services Tutorial
- Microsoft Azure Tutorial
- Git Tutorial
- Ethical Hacking Tutorial
- Docker Tutorial
- Kubernetes Tutorial
- DSA Tutorial
- Spring Boot Tutorial
- SDLC Tutorial
- Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification



Chapters ▾

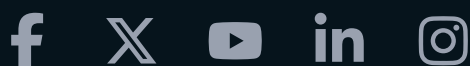
Categories ≡

Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)





Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.